

HIGH-PERFORMANCE AND CLOUD COMPUTING

Unit 2: OpenMP and MPI

Writing parallel programs for shared memory with OpenMP and for distributed memory with MPI, and combining the two.

Prepared by **Dr. Abhijith Anandakrishnan**

Assistant Professor, Amrita School of AI

COURSE

23AID304 High-Performance and Cloud Computing

UNIT

2 of 4

SYLLABUS

OpenMP essentials, data sharing and synchronisation, efficient OpenMP for matrix computing, MPI and distributed memory, communicating with MPI, parallel matrix solvers, domain decomposition

PREREQUISITE

Units 0 and 1

OFFERING

Odd Semester 2026-27

The two models, side by side

Unit 1 established the distinction. This unit makes it concrete.

	OPENMP	MPI
Full name	Open Multi-Processing	Message Passing Interface
Model	Shared memory, threads	Distributed memory, processes
What it is	A compiler extension: directives plus a small runtime library	A library specification with several implementations
Parallel unit	Thread	Rank (a process)
How data moves	Threads read the same variables	Explicit messages
Adding it to serial code	Incremental, a loop at a time	Usually a restructure
Scales to	One node	The whole cluster
Current version	OpenMP 6.0 (2024)	MPI 5.0 (2025)
The characteristic bug	Data race	Deadlock

Neither replaces the other. On `asaicompute`, OpenMP uses the 96 cores of one node, MPI spans the three nodes, and the two together are how you use the whole machine. That combination is the last section of this unit.

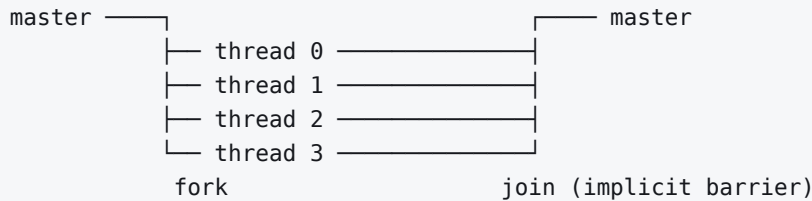
BY THE END OF THIS UNIT YOU WILL BE ABLE TO

- Write correct OpenMP programs using parallel regions, worksharing, and reductions.
- Choose the right data-sharing clause and explain what each one does.
- Recognise, reproduce, and fix a data race and a false-sharing slowdown.
- Choose a loop schedule based on the shape of the work.
- Write MPI programs using point-to-point and collective communication.
- Explain why the obvious send-receive pattern deadlocks, and fix it three ways.
- Decompose a grid problem into subdomains with halo exchange.
- Combine MPI and OpenMP in one program and say when it is worth doing.

Part I: OpenMP

The model

OpenMP uses **fork-join** parallelism. The program runs on one thread, called the master. At a parallel region it forks a team of threads; at the end of the region they join and the program continues serially.



You express parallelism with **directives**, which are pragmas the compiler interprets. Without `-fopenmp` the compiler ignores every one of them and produces a correct serial program. That property is genuinely useful: the same source compiles and runs either way.

```
#include <omp.h>

#pragma omp parallel
{
    int id = omp_get_thread_num(); /* 0 .. nthreads-1 */
    int n = omp_get_num_threads();
    printf("thread %d of %d\n", id, n);
} /* implicit barrier here */
```

Compile and run:

```
gcc -O2 -fopenmp prog.c -o prog
OMP_NUM_THREADS=8 ./prog
```

PITFALL · THE SINGLE MOST COMMON OPENMP MISTAKE

Forgetting `-fopenmp`. The program compiles without error, runs on one thread, and `omp_get_num_threads()` returns 1. There is no warning. If your parallel program shows no speedup at all, check the compile line before anything else.

Worksharing

A bare `parallel` region makes every thread run the *same* code. To split a loop across the team, add a worksharing construct.

```
#pragma omp parallel for
for (int i = 0; i < n; i++) {
    a[i] = b[i] + c[i];
}
```

The loop must be in **canonical form**: a countable loop whose trip count the runtime can compute before it starts. So `for (i = 0; i < n; i++)` is fine, and a `while` loop or a loop with `break` is not.

PITFALL · LOOP-CARRIED DEPENDENCES

```
#pragma omp parallel for          /* WRONG */
for (i = 1; i < n; i++) a[i] = a[i-1] + b[i];
```

Iteration i needs the result of iteration $i-1$. The compiler will happily parallelise it and the answer will be wrong. OpenMP does not check for dependences; that is your job. This particular pattern, a prefix sum, has a genuinely parallel algorithm, but it is not this loop.

Other worksharing constructs:

```
#pragma omp parallel
{
    #pragma omp sections          /* independent blocks of work */
    {
        #pragma omp section
        do_left_half();
        #pragma omp section
        do_right_half();
    }

    #pragma omp single            /* exactly one thread; others wait */
    printf("done\n");

    #pragma omp master            /* thread 0 only; NO barrier */
    checkpoint();
}
```

DEFINITION · SINGLE VERSUS MASTER

`single` means *one* thread, not necessarily thread 0, and it ends with an implicit barrier. `master` means specifically thread 0 and has **no** barrier, so the others race straight past it. Choosing `master` when you needed `single` is a subtle and common bug.

Data sharing: the part that causes the bugs

Every variable in a parallel region is either shared by all threads or private to each. Getting this wrong is the source of almost all OpenMP defects.

CLAUSE	MEANING
<code>shared(x)</code>	One copy, all threads see it. Default for variables declared outside the region
<code>private(x)</code>	Each thread gets its own uninitialised copy
<code>firstprivate(x)</code>	Private, initialised to the value before the region
<code>lastprivate(x)</code>	Private, and the value from the <i>sequentially last</i> iteration is copied out
<code>reduction(op:x)</code>	Private copy initialised to the operator's identity, all copies combined at the end
<code>default(none)</code>	Forces you to classify every variable explicitly

PITFALL · PRIVATE DOES NOT INITIALISE

```
int t = 5;
#pragma omp parallel private(t)
printf("%d\n", t);          /* prints garbage, not 5 */
```

`private` gives each thread fresh, uninitialised storage. If you want the incoming value, use `firstprivate`.

DEFINITION · USE DEFAULT(NONE)

```
#pragma omp parallel for default(none) shared(a, b, n) private(tmp)
```

With `default(none)` the compiler refuses to guess, and any variable you forgot becomes a compile error rather than a silent race. Every OpenMP program in this course should use it. It turns a debugging session into a compiler message.

Reductions

The pattern from Unit 0's exercise, stated properly. A reduction combines a value across all iterations:

```
double sum = 0.0;
#pragma omp parallel for reduction(+ : sum)
for (int i = 0; i < n; i++) sum += a[i];
```

Each thread gets a private `sum` initialised to the identity of the operator, and the runtime combines them at the end. Supported operators are `+ - * & | ^ && || max min`, and OpenMP also allows user-defined reductions with `#pragma omp declare reduction`.

COMMON MISCONCEPTION · REDUCTION AND CRITICAL ARE TWO WAYS TO DO THE SAME THING

They produce the same answer and have completely different costs.

```
#pragma omp parallel for          /* SLOW */  
for (i = 0; i < n; i++)  
    #pragma omp critical  
    sum += a[i];
```

Here every iteration takes a lock, so the loop is fully serialised *plus* lock overhead, and it is reliably slower than the serial version. A reduction has no per-iteration synchronisation at all: it costs one combination step of $\log(\text{threads})$ at the very end.

Synchronisation

```
#pragma omp barrier      /* everyone waits here */  
#pragma omp critical     /* one thread at a time, whole block */  
#pragma omp atomic       /* one memory update, uses a hardware instruction */  
#pragma omp nowait       /* remove the implicit barrier at a worksharing end */
```

`atomic` is much cheaper than `critical` but applies only to a single read-modify-write of one memory location. Use it where it fits:

```
#pragma omp atomic  
histogram[bin]++;          /* fine: one update, unpredictable index */
```

PITFALL · BARRIERS MUST BE REACHED BY EVERYONE

```
#pragma omp parallel  
{  
    if (omp_get_thread_num() == 0) {  
        #pragma omp barrier      /* DEADLOCK */  
    }  
}
```

Every thread in the team must execute the same barrier. One thread waiting for others that will never arrive hangs the program forever.

False sharing

This is the bug that teaches more about hardware than any lecture.

```
double partial[NTHREADS];           /* SLOW */
#pragma omp parallel
{
    int id = omp_get_thread_num();
    for (long i = id; i < n; i += NTHREADS)
        partial[id] += a[i];
}
```

There is **no data race**: each thread writes only its own element, and the answer is always correct. But `partial[0]` through `partial[7]` are 8 bytes apart, so all eight sit inside a single 64-byte cache line. Every write by any thread invalidates that line in every other core's cache. The line pings-pongs across the interconnect thousands of times a second, and the loop can run several times **slower** than serial.

DEFINITION · FALSE SHARING

A performance collapse caused by several cores writing to *different* variables that happen to share one cache line. There is no correctness bug, which is precisely what makes it hard to find: the program is right, and slow, and nothing in the source looks wrong.

Three fixes, in increasing order of preference:

```
/* 1. Pad so each thread's accumulator owns a whole cache line */
typedef struct { double v; char pad[56]; } padded_t; /* 64 bytes total */
padded_t partial[NTHREADS];

/* 2. Accumulate in a local variable, write once at the end */
#pragma omp parallel
{
    double local = 0.0; /* lives in a register */
    for (long i = id; i < n; i += NTHREADS) local += a[i];
    partial[id] = local; /* one write, not a billion */
}

/* 3. Just use a reduction, which does exactly this for you */
#pragma omp parallel for reduction(+: sum)
for (long i = 0; i < n; i++) sum += a[i];
```

WORKED EXAMPLE · DIAGNOSING FALSE SHARING

Symptoms: the parallel version is slower than serial; the slowdown gets *worse* with more threads; the answer is always correct; there is no lock in sight.

Confirm it by padding the array and re-measuring. If the time collapses, that was it. On Linux you can also see it directly:

```
perf stat -e cache-misses,cache-references ./prog
```

A false-sharing loop shows an absurd cache-miss count for the amount of data it touches.

Scheduling

```
#pragma omp parallel for schedule(static)           /* equal contiguous chunks */
#pragma omp parallel for schedule(static, 16)       /* round-robin, 16 at a time */
#pragma omp parallel for schedule(dynamic, 8)       /* take 8 when free */
#pragma omp parallel for schedule(guided)           /* big chunks first, shrinking */
#pragma omp parallel for schedule(runtime)          /* read OMP_SCHEDULE at run time */
```

SCHEDULE	OVERHEAD	USE WHEN
static	Lowest: computed once, no runtime coordination	Every iteration costs about the same
dynamic	Highest: threads coordinate on every chunk	Iteration cost varies a lot or unpredictably
guided	Middle	Cost varies moderately; good default when unsure

WORKED EXAMPLE · WHEN SCHEDULE MATTERS: A TRIANGULAR LOOP

```
#pragma omp parallel for schedule(static)
for (i = 0; i < n; i++)
    for (j = i; j < n; j++) /* inner trip count falls with i */
        work(i, j);
```

With `static`, thread 0 gets the first quarter of i , which contains far more inner iterations than the last quarter. Thread 0 does roughly seven times the work of thread 3, and everyone waits for it: efficiency around 0.4 on four threads.

Changing one word to `schedule(dynamic, 16)` typically takes efficiency above 0.9. Nothing else in the program changes.

Note the chunk size. `schedule(dynamic)` alone means chunk size 1, and for cheap iterations the coordination cost then exceeds the work. Pick a chunk large enough to amortise the overhead and small enough to balance.

Tasks

Loops cover regular work. **Tasks** cover recursion, trees, and linked structures.

```
#pragma omp parallel
#pragma omp single /* one thread creates the tasks */
{
    #pragma omp task shared(x)
    x = solve_left();
    #pragma omp task shared(y)
    y = solve_right();
    #pragma omp taskwait /* wait for both */
    combine(x, y);
}
```

WORKED EXAMPLE · PARALLEL MERGE SORT WITH A CUT-OFF

```
void psort(int *a, int lo, int hi, int depth)
{
    if (hi - lo < 10000 || depth <= 0) { qsort_serial(a, lo, hi); return; }
    int mid = (lo + hi) / 2;
    #pragma omp task shared(a)
    psort(a, lo, mid, depth - 1);
    #pragma omp task shared(a)
    psort(a, mid, hi, depth - 1);
    #pragma omp taskwait
    merge(a, lo, mid, hi);
}
```

The cut-off is essential. Creating a task costs roughly a microsecond, so spawning one to sort four elements is pure loss. Stop recursing when either the problem is small enough or the tree is deep enough. Choosing that threshold is the granularity decision from Unit 1, in code.

BEYOND THE SYLLABUS · WHAT OPENMP 6.0 ADDED

The current specification (November 2024) brings **free-agent threads**, so tasks can be executed by threads that are not part of the generating team; the ability to **record and replay task graphs**, which removes the repeated scheduling cost from an iterative solver; and new loop transformation directives such as `tile`, `unroll`, `fuse` and `flatten` that let you request transformations explicitly instead of hoping the compiler applies them. It also removes everything deprecated in 5.x. Compiler support is still arriving; check what your `gcc` implements before relying on any of it.

Efficient OpenMP for matrix computation

The syllabus asks specifically for this, and it is where Units 0, 1 and 2 meet.

The naive version

```
for (i = 0; i < n; i++)
  for (j = 0; j < n; j++) {
    double s = 0.0;
    for (k = 0; k < n; k++) s += A[i][k] * B[k][j];
    C[i][j] = s;
  }
```

`B[k][j]` walks **down a column**, so it strides n doubles between accesses. That is Figure 0.2 in Unit 0: one useful double per cache line. This is why the naive triple loop sits at $I \approx 0.25$ on the roofline and achieves a few percent of peak.

Step 1: fix the loop order

Swapping the two inner loops turns the column walk into a row walk:

```
for (i = 0; i < n; i++)
  for (k = 0; k < n; k++) {
    double aik = A[i][k];
    for (j = 0; j < n; j++) C[i][j] += aik * B[k][j];    /* both walk rows */
  }
```

Same flops, same answer, typically **three to five times faster**, and now it vectorises because the inner loop is a contiguous DAXPY.

Step 2: block for cache

Loop order fixes spatial locality. Blocking fixes *capacity*: it makes the working set small enough to stay in cache while it is being reused.

```
for (ii = 0; ii < n; ii += BS)
  for (kk = 0; kk < n; kk += BS)
    for (jj = 0; jj < n; jj += BS)
      for (i = ii; i < min(ii+BS, n); i++)
        for (k = kk; k < min(kk+BS, n); k++) {
          double aik = A[i][k];
          for (j = jj; j < min(jj+BS, n); j++)
            C[i][j] += aik * B[k][j];
        }
```

Three `BS × BS` blocks must fit in cache together, so $3 \cdot BS^2 \cdot 8 \text{ bytes} \leq \text{cache size}$. For a 256 KB L2, $BS \approx 100$; values of 32 to 128 are typical, and the best one is found by measurement.

As Unit 1 showed, blocking raises arithmetic intensity from about 0.25 to about $BS/4$, which moves the kernel from far left of the ridge point to the right of it. That is the whole reason it works.

Step 3: parallelise

```
#pragma omp parallel for collapse(2) schedule(static)
for (ii = 0; ii < n; ii += BS)
    for (jj = 0; jj < n; jj += BS)
        ...
```

`collapse(2)` merges two loop nests into one iteration space, giving the scheduler more chunks to distribute. Parallelise the **outer** loops: parallelising the innermost one would fork and join a team on every pass, and the overhead would swamp the work.

WORKED EXAMPLE · THE FULL SEQUENCE ON A 2048 × 2048 MATRIX

Indicative figures on one node, double precision. Your numbers will differ; the *pattern* will not.

VERSION	TIME	SPEEDUP OVER NAIVE	PERCENT OF PEAK
Naive <code>ijk</code>	42 s	1x	0.4%
Reordered <code>ikj</code>	9.8 s	4.3x	1.8%
Blocked, BS = 64	3.1 s	13.5x	5.5%
Blocked + 16 threads	0.24 s	175x	71%
Vendor BLAS <code>dgemv</code>	0.11 s	380x	155%*

* Above 100 percent of our nominal peak because the library uses fused multiply-add, counted as two flops in one instruction.

Two lessons. First, **the serial optimisations came before the parallel one and were worth 13.5x on their own**; parallelising the naive version instead would have given 16 cores all waiting on memory. Second, a tuned library still beats hand-written code by a factor of two, which is why the right answer in production is almost always to call BLAS.

PITFALL · DO NOT WRITE YOUR OWN GEMM FOR PRODUCTION

This exercise is to teach you *why* blocking works. Real code should call an optimised BLAS: OpenBLAS, Intel MKL, AMD AOCL, or cuBLAS on a GPU. Those libraries are hand-tuned per microarchitecture, use every register, and are tested to death. Understanding the technique is the point; reimplementing it is not.

Part II: MPI

The model

An MPI program runs as P independent processes, each with its own private memory. They are started together, they all run the same executable, and each learns its own identity at startup. That is SPMD from Unit 1.

```
#include <mpi.h>

int main(int argc, char **argv)
{
    MPI_Init(&argc, &argv);

    int rank, size;
    MPI_Comm_rank(MPI_COMM_WORLD, &rank); /* who am I: 0 .. size-1 */
    MPI_Comm_size(MPI_COMM_WORLD, &size); /* how many of us */

    printf("rank %d of %d\n", rank, size);

    MPI_Finalize();
    return 0;
}
```

```
mpicc -O2 prog.c -o prog
mpirun -np 4 ./prog
```

DEFINITION · COMMUNICATOR, RANK, SIZE

A **communicator** is a group of processes that can talk to each other. `MPI_COMM_WORLD` contains all of them. A **rank** is a process's index within a communicator, from 0 to size-1. A process can belong to several communicators and will generally have a different rank in each.

Point-to-point communication

```
MPI_Send(buf, count, datatype, dest, tag, comm);
MPI_Recv(buf, count, datatype, source, tag, comm, &status);
```

The six arguments that matter: **what** (buffer, count, datatype), **who** (dest or source), and **which message** (tag, communicator). A receive matches a send only when source, tag and communicator all agree.

DATATYPE	C TYPE
<code>MPI_INT</code>	<code>int</code>
<code>MPI_DOUBLE</code>	<code>double</code>
<code>MPI_FLOAT</code>	<code>float</code>
<code>MPI_CHAR</code>	<code>char</code>
<code>MPI_BYTE</code>	raw bytes, no conversion

Wildcards `MPI_ANY_SOURCE` and `MPI_ANY_TAG` accept from anyone; the actual sender is then read from `status.MPI_SOURCE`.

The deadlock everyone writes

```
/* Exchange boundary values with a neighbour. DEADLOCKS. */
MPI_Send(sendbuf, n, MPI_DOUBLE, neighbour, 0, MPI_COMM_WORLD);
MPI_Recv(recvbuf, n, MPI_DOUBLE, neighbour, 0, MPI_COMM_WORLD, &status);
```

Both processes call `MPI_Send` first. `MPI_Send` is allowed to block until the matching receive is posted. Neither ever reaches its `MPI_Recv`. The program hangs.

PITFALL · IT WORKED WITH SMALL MESSAGES, SO I THOUGHT IT WAS FINE

For small messages most implementations copy into an internal buffer and return immediately, so the code appears to work. Cross the **eager limit**, typically a few tens of kilobytes, and the implementation switches to a rendezvous protocol that genuinely blocks. The program then deadlocks on a larger problem size, in production, having passed every test. Never rely on buffering.

Three correct fixes:

```
/* 1. Order the operations by rank: odd sends first, even receives first */
if (rank % 2 == 0) { MPI_Send(...); MPI_Recv(...); }
else                { MPI_Recv(...); MPI_Send(...); }

/* 2. Sendrecv: one call, the implementation guarantees no deadlock */
MPI_Sendrecv(sendbuf, n, MPI_DOUBLE, neighbour, 0,
             recvbuf, n, MPI_DOUBLE, neighbour, 0,
             MPI_COMM_WORLD, &status);

/* 3. Non-blocking: post both, then wait. Also allows overlap */
MPI_Request req[2];
MPI_Irecv(recvbuf, n, MPI_DOUBLE, neighbour, 0, MPI_COMM_WORLD, &req[0]);
MPI_Isend(sendbuf, n, MPI_DOUBLE, neighbour, 0, MPI_COMM_WORLD, &req[1]);
compute_interior(); /* real work while data flies */
MPI_Waitall(2, req, MPI_STATUSES_IGNORE);
```

Option 3 is the one used in serious code, because it does more than avoid deadlock: it lets you overlap communication with computation, hiding the network latency behind useful work.

PITFALL · NON-BLOCKING MEANS THE BUFFER IS NOT YOURS YET

After `MPI_Isend` returns, the message has not been sent. Modifying `sendbuf` before `MPI_Wait` completes corrupts the message, usually intermittently and on someone else's machine. The same applies to reading `recvbuf` after `MPI_Irecv`.

Collective communication

Collectives are called by **every** rank in the communicator, and they are both clearer and faster than hand-rolled equivalents, because implementations use tree and ring algorithms tuned to the network.

CALL	WHAT IT DOES
<code>MPI_Barrier</code>	Everyone waits for everyone
<code>MPI_Bcast</code>	One rank's data goes to all
<code>MPI_Scatter</code>	Root splits an array, one piece per rank
<code>MPI_Gather</code>	Inverse of scatter: pieces collected at root
<code>MPI_Allgather</code>	Gather, with the result given to everyone
<code>MPI_Reduce</code>	Combine values from all ranks at the root
<code>MPI_Allreduce</code>	Reduce, with the result given to everyone
<code>MPI_Alltoall</code>	Every rank sends a distinct piece to every other

Almost every MPI program is the same three movements: **distribute** the data, **compute** locally, **combine** the results.

WORKED EXAMPLE · DISTRIBUTED DOT PRODUCT

```
int rank, size;
MPI_Comm_rank(MPI_COMM_WORLD, &rank);
MPI_Comm_size(MPI_COMM_WORLD, &size);

int local_n = N / size; /* assume divisible for clarity */
double *xa = malloc(local_n * sizeof *xa);
double *ya = malloc(local_n * sizeof *ya);

MPI_Scatter(x, local_n, MPI_DOUBLE, xa, local_n, MPI_DOUBLE, 0, MPI_COMM_WORLD);
MPI_Scatter(y, local_n, MPI_DOUBLE, ya, local_n, MPI_DOUBLE, 0, MPI_COMM_WORLD);

double local = 0.0;
for (int i = 0; i < local_n; i++) local += xa[i] * ya[i];

double total;
MPI_Allreduce(&local, &total, 1, MPI_DOUBLE, MPI_SUM, MPI_COMM_WORLD);
/* every rank now has the full dot product */
```

Note that `MPI_Allreduce` on P ranks costs about $\log_2 P$ communication steps, not P . Writing this by hand as P sends to rank 0 would be P steps and would also serialise on rank 0's network link.

COMMON MISCONCEPTION · COLLECTIVES ARE JUST CONVENIENCE WRAPPERS

They are the opposite: they are the place implementations put their best algorithms. `MPI_Allreduce` may use a recursive-halving reduce-scatter followed by a recursive-doubling allgather, switching strategy by message size and topology. Hand-written equivalents are almost always slower, and a great deal longer.

Derived datatypes

Sending a column of a C matrix means sending elements that are not adjacent in memory. Rather than copying them into a temporary buffer by hand, describe the pattern once:

```
MPI_Datatype column;
MPI_Type_vector(nrows, /* number of blocks */
                1, /* elements per block */
                ncols, /* stride between blocks */
                MPI_DOUBLE, &column);
MPI_Type_commit(&column);

MPI_Send(&A[0][j], 1, column, dest, 0, MPI_COMM_WORLD);
MPI_Type_free(&column);
```

This lets the implementation move the data in the way that suits the hardware, and it keeps the intent visible in the code.

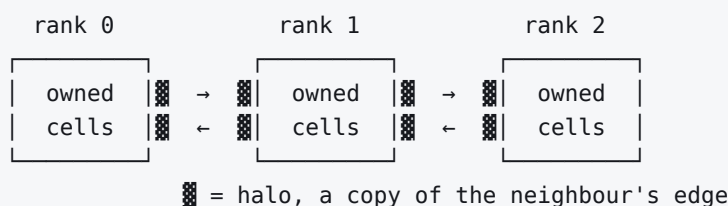
BEYOND THE SYLLABUS · MPI 5.0 AND THE STANDARD ABI

MPI 5.0, published in June 2025, is the first version to define a standard **ABI**, an application binary interface. Until now a program compiled against Open MPI could not run against MPICH without recompiling, because the two implementations laid out MPI handles differently. With a standard ABI a binary can work with any compatible implementation, which matters a great deal for distributing compiled scientific software and containers. MPI 5.0 also completes **Sessions**, an alternative to `MPI_Init` that allows a library to use MPI without assuming it owns the whole program, and adds internal datatype mapping. MPICH 5.0.0 shipped support in February 2026.

Domain decomposition and halo exchange

This is the pattern behind most distributed scientific computing, and it is the PCAM worked example from Unit 1 realised in code.

Split a grid across ranks. Each rank owns a subdomain, but to update its edge cells it needs its neighbours' edge values. So each rank allocates a border of extra cells, called the **halo** or **ghost** region, and refreshes it each step.



```
/* One time step of a 1-D stencil, with overlap */
MPI_Request req[4];
int left  = (rank == 0)      ? MPI_PROC_NULL : rank - 1;
int right = (rank == size - 1) ? MPI_PROC_NULL : rank + 1;

MPI_Irecv(&u[0],          1, MPI_DOUBLE, left,  0, comm, &req[0]);
MPI_Irecv(&u[local_n+1], 1, MPI_DOUBLE, right, 1, comm, &req[1]);
MPI_Isend(&u[1],          1, MPI_DOUBLE, left,  1, comm, &req[2]);
MPI_Isend(&u[local_n],    1, MPI_DOUBLE, right, 0, comm, &req[3]);

/* interior cells need no halo, so update them while messages are in flight */
for (int i = 2; i <= local_n - 1; i++)
    unew[i] = 0.5 * (u[i-1] + u[i+1]);

MPI_Waitall(4, req, MPI_STATUSES_IGNORE);

/* now the two edge cells */
unew[1]      = 0.5 * (u[0]      + u[2]);
unew[local_n] = 0.5 * (u[local_n-1] + u[local_n+1]);
```

`MPI_PROC_NULL` is a convenience worth knowing: sends to it and receives from it succeed immediately and do nothing, which removes every boundary special case from the code.

WORKED EXAMPLE · SURFACE-TO-VOLUME, AND WHY 2-D BEATS 1-D

Split an $N \times N$ grid over P ranks.

1-D strips: each rank owns N/P rows, so it computes N^2/P cells and communicates $2N$ halo values. Ratio = $2P/N$.

2-D blocks: each rank owns an $(N/\sqrt{P}) \times (N/\sqrt{P})$ block, computes N^2/P cells, and communicates $4N/\sqrt{P}$ values. Ratio = $4\sqrt{P}/N$.

With $N = 1024$ and $P = 64$: 1-D gives 0.125, 2-D gives 0.031, a **four times** lower communication cost for identical computation. The advantage grows as $\sqrt{P}$, so at large scale 2-D or 3-D decomposition is not an optimisation, it is a requirement. This is the *surface-to-volume ratio* argument, and it is the reason real solvers decompose in every available dimension.

Parallel matrix solvers, briefly

The syllabus asks for parallel matrix solvers and matrix representations of physical systems. The connection is that discretising a physical problem, heat flow, elasticity, electrostatics, produces a large sparse linear system $Ax = b$.

Two families:

- **Direct** methods, such as LU and Cholesky factorisation. Predictable cost, but factorisation creates **fill-in**: entries that were zero become non-zero, so memory grows and the communication pattern becomes irregular. Parallel direct solvers are hard, and libraries such as MUMPS and SuperLU exist because of it.
- **Iterative** methods, such as Jacobi, Gauss-Seidel, conjugate gradient. Each iteration is a sparse matrix-vector product plus a few dot products. This parallelises beautifully: the matrix-vector product is a halo exchange, and the dot products are `MPI_Allreduce`. This is why almost all large-scale solvers are iterative.

Conjugate gradient, one iteration, in parallel terms:

$q = A \cdot p$	→ halo exchange, then local sparse matvec
$\alpha = p / (p \cdot q)$	→ <code>MPI_Allreduce</code> (one scalar)
$x = x + \alpha \cdot p$	→ purely local
$r = r - \alpha \cdot q$	→ purely local
$\rho_{\text{new}} = r \cdot r$	→ <code>MPI_Allreduce</code> (one scalar)

Two global reductions per iteration. At very large scale those two `Allreduce` calls become the bottleneck, which is why *communication-avoiding* Krylov methods are an active research area.

Part III: Hybrid MPI + OpenMP

BEYOND THE SYLLABUS · NOT IN THE SYLLABUS, AND UNAVOIDABLE IN PRACTICE

The syllabus treats OpenMP and MPI separately. Every real cluster application combines them, because a modern node has 96 cores and running 96 separate MPI ranks on it wastes memory and multiplies message counts.

The standard arrangement: **one MPI rank per node or per NUMA domain, OpenMP threads within.**

```
int provided;
MPI_Init_thread(&argc, &argv, MPI_THREAD_FUNNELED, &provided);
if (provided < MPI_THREAD_FUNNELED) { /* handle it */ }

#pragma omp parallel for
for (i = 0; i < local_n; i++) compute(i);

MPI_Allreduce(...);          /* only the master thread calls MPI */
```

THREAD LEVEL	MEANING
<code>MPI_THREAD_SINGLE</code>	One thread only
<code>MPI_THREAD_FUNNELED</code>	Only the thread that called <code>MPI_Init_thread</code> may call MPI
<code>MPI_THREAD_SERIALIZED</code>	Any thread may, but never two at once
<code>MPI_THREAD_MULTIPLE</code>	Any thread, any time. Convenient, often slower

Request the weakest level that works. `MPI_THREAD_MULTIPLE` forces the implementation to add internal locking, and it frequently costs more than it saves.

Why hybrid helps: fewer ranks means fewer and larger messages (recall $T = \alpha + n/\beta$, where α is per message), halo regions are shared rather than duplicated per rank, and a single copy of read-only data serves all threads on the node.

```
# 3 nodes, 1 rank each, 96 threads per rank
export OMP_NUM_THREADS=96
export OMP_PROC_BIND=close
export OMP_PLACES=cores
srun --nodes=3 --ntasks-per-node=1 --cpus-per-task=96 ./hybrid
```

ON ASACOMPUTE · THREAD PINNING IS NOT OPTIONAL

Without `OMP_PROC_BIND`, Linux may migrate a thread to another core mid-run. The thread arrives with cold caches and, on a NUMA machine, possibly on the wrong side of the memory it is using. Set

`OMP_PROC_BIND=close` and `OMP_PLACES=cores` for every measurement in this course. Unpinned timings are not reproducible and will not be accepted in your scaling studies.

Self-assessment

1. What does `#pragma omp parallel` do on its own, without `for`? How many times does the enclosed code run?
2. Explain the difference between `private`, `firstprivate` and `lastprivate`.
3. Why should every OpenMP directive in this course use `default(none)`?
4. Rewrite correctly, and explain the bug: `#pragma omp parallel for over for (i=0;i<n;i++) { t = a[i]*2; b[i] = t+1; }` where `t` is declared outside the loop.
5. A loop using `#pragma omp critical` to accumulate a sum runs slower than serial. Explain why, and give the correct construct.
6. What is false sharing? Why is it hard to find? Give two fixes.
7. When would you choose `schedule(dynamic, 32)` over `schedule(static)`?
8. Why does a task-based recursive algorithm need a cut-off?
9. Multiplying matrices, why is reordering the loops to `ikj` faster than `ijk`, given both do the same number of operations?
10. What does blocking change, if not the flop count? Relate it to Unit 1.
11. Write the four lines that every MPI program starts and ends with.
12. Explain precisely why the send-then-receive exchange deadlocks, and why it may appear to work in testing.
13. Give three ways to fix that deadlock, and say which is best and why.
14. After `MPI_Isend` returns, may you modify the send buffer? Explain.
15. On 64 ranks, how many communication steps does a good `MPI_Allreduce` take, and how many would a naive gather-at-rank-0 take?
16. What is a halo region and why is it needed?
17. For a 2048×2048 grid on 256 ranks, compute the communication-to-computation ratio for 1-D and 2-D decomposition. Which would you choose?
18. Why are iterative solvers preferred over direct solvers at large scale?
19. What is `MPI_THREAD_FUNNELED` and why not always request `MPI_THREAD_MULTIPLE`?
20. Give two reasons one rank per node with 96 threads may beat 96 ranks per node.

-
1. It creates a team of threads, and **every** thread executes the whole enclosed block. With 8 threads the code runs 8 times. `for` is what *divides* work; `parallel` alone only *replicates* it.
 2. `private` gives each thread its own uninitialised copy. `firstprivate` gives each thread its own copy initialised to the value the variable had before the region. `lastprivate` copies the value from the sequentially last iteration back out to the original variable after the region.
 3. Because the default is `shared` for anything declared outside the region, so a forgotten variable becomes a silent data race. `default(none)` turns that omission into a compile error.
 4. `t` is shared, so all threads write the same variable and `b[i]` gets whichever value happened to be there. Fix by declaring `t` inside the loop body, or by adding `private(t)`. Declaring it inside is preferable: it makes the intent obvious without a clause.
 5. `critical` serialises the loop: every iteration acquires and releases a lock, so no two iterations overlap, and you pay the lock cost on top of the serial work. Use `reduction(+:sum)`, which gives each thread a private accumulator and combines them once at the end.
 6. Several cores writing to different variables that share a 64-byte cache line; each write invalidates the line in the other cores' caches and it ping-pongs. It is hard to find because there is no correctness bug and nothing in the source looks wrong; the only symptom is that it is slow and gets worse with more threads. Fixes: pad each thread's data to a full cache line, or accumulate into a thread-local variable and write once at the end (a reduction does exactly this).
 7. When iteration cost varies substantially or unpredictably, for example a triangular loop, an early-exit search, or work that depends on data values. The chunk size 32 amortises the coordination overhead that plain `dynamic` (chunk 1) would incur.
 8. Task creation costs roughly a microsecond. Below some problem size the overhead exceeds the work, so recursion must switch to a serial algorithm at a threshold. Without one, a recursion to single elements spends nearly all its time in the runtime.
 9. In `ijk`, the inner loop indexes `B[k][j]` with k varying, which strides a whole row between accesses, using 8 bytes of every 64-byte line. In `ikj`, the inner loop indexes both `C[i][j]` and `B[k][j]` with j varying, which is contiguous, so all 8 doubles of each line are used and the loop also vectorises. Same arithmetic, roughly eight times less memory traffic.
 10. It changes Q , the number of bytes that must be fetched from DRAM, by reusing each loaded block b times before evicting it. Since $I = W/Q$ and W is unchanged, arithmetic intensity rises by roughly a factor of b , moving the kernel to the right on the roofline from memory bound towards compute bound.
 11. `MPI_Init(&argc,&argv)`, `MPI_Comm_rank(MPI_COMM_WORLD,&rank)`, `MPI_Comm_size(MPI_COMM_WORLD,&size)`, and `MPI_Finalize()`.
 12. Both processes call `MPI_Send` first, and `MPI_Send` is permitted to block until a matching receive is posted. Each waits for the other's receive, which is never reached. It appears to work in testing because small messages are copied into an internal buffer and the send returns immediately; above the eager limit the implementation switches to rendezvous and genuinely blocks.
 13. Order operations by rank parity; use `MPI_Sendrecv`; or use `MPI_Irecv` and `MPI_Isend` followed by `MPI_Waitall`. The non-blocking form is best because it avoids deadlock *and* allows interior computation to overlap the communication, hiding network latency.

14. No. `MPI_Isend` only *initiates* the transfer; the buffer belongs to MPI until the matching `MPI_Wait` or `MPI_Test` reports completion. Modifying it before then corrupts the message, usually intermittently.
15. A good `MPI_Allreduce` takes about $\log_2 64 = 6$ steps. A naive gather at rank 0 takes **63** sends and also serialises on rank 0's single network link.
16. A border of extra cells holding copies of the neighbouring subdomain's edge values. It is needed because updating a cell at the edge of a subdomain requires values that another rank owns, and refreshing a halo once per step is far cheaper than communicating per cell.
17. 1-D: $2P/N = 2 \times 256 / 2048 = \mathbf{0.25}$. 2-D: $4\sqrt{P}/N = 4 \times 16 / 2048 = \mathbf{0.031}$. The 2-D decomposition communicates about **eight times less** for the same computation, so choose 2-D.
18. Direct factorisation causes fill-in, so memory grows well beyond the original sparse matrix and the communication pattern becomes irregular and hard to balance. Iterative methods keep the matrix untouched, and each iteration reduces to a halo exchange plus a couple of `Allreduce` calls, both of which parallelise predictably.
19. It declares that only the thread which called `MPI_Init_thread` will make MPI calls. Requesting `MPI_THREAD_MULTIPLE` forces the implementation to add internal locking on every MPI call to make it thread-safe, which commonly costs more performance than the flexibility is worth. Request the weakest level that works.
20. Fewer, larger messages, so less of the per-message latency α ; and halo regions and read-only data are shared once per node rather than duplicated per rank, saving memory and the work of keeping copies consistent.

Where to go next

Unit 3 takes the same ideas to the GPU. SIMD becomes SIMT, threads become a hierarchy of thread blocks and warps, cache blocking becomes shared-memory tiling, and coalesced access is the cache-line argument of Unit 0 in new clothing. If you understood why `ikj` beats `ijk`, you already understand why coalescing matters.